Top 30 Python Coding Questions

★ Q1. Reverse a String

Input : hello

Output : olleh

Algorithm :

- Use Python slicing.
- Reverse from last index to first.
- Return reversed string.

Time Complexity : O(n)

Space Complexity : O(1)

Code Solution :

```
def reverse_string(s):
    return s[::-1]

print(reverse_string("hello"))

# Output : olleh
```

★ Q2. Check if a String is a Palindrome

Input : Madam

Output : True

Algorithm :

- Convert string to lowercase and remove spaces.
- Compare the string with its reverse.
- If both are same, return True, otherwise False.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

print(is_palindrome("Madam"))

# Output : True
```

★ Q3. FizzBuzz

Input : n = 15

Output : 1 2 Fizz 4 Buzz ... 14 FizzBuzz

Algorithm :

- Iterate from 1 to n.
- If number divisible by 15 → FizzBuzz.
- Else if divisible by 3 → Fizz.
- Else if divisible by 5 → Buzz.
- Otherwise → print number.

Time Complexity : O(n)

Space Complexity : O(1)

Code Solution :

```
def fizzbuzz(n):
    for i in range(1, n + 1):
        if i % 15 == 0:
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)
```

```
# Output for n = 15 :
1 2 Fizz 4 Buzz | ... | 14 FizzBuzz
```

Top 30 Python Coding Questions

★ Q4. Find Duplicates in a List

Input : [1, 2, 3, 2, 4, 1]

Output : [1, 2]

Algorithm :

- Iterate through list using a set to track seen elements.
- If element already seen, add to duplicates set.
- Return list of duplicates.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def find_duplicates(lst):
    seen, dupes = set(), set()
    for x in lst:
        if x in seen:
            dupes.add(x)
        seen.add(x)
    return list(dupes)

print(find_duplicates([1, 2, 3, 2, 4, 1]))
# Output : [1, 2]
```

★ Q5. Two Sum

Input : nums = [2, 7, 11, 15], target = 9

Output : [0, 1]

Algorithm :

- Use a dictionary to store number and its index.
- For each number, check if (target - num) exists in dictionary.
- If yes, return the pair of indices.
- Otherwise, store current number and index.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return []

print(two_sum([2, 7, 11, 15], 9))
# Output : [0, 1]
```

★ Q6. Fibonacci Series (Iterative)

Input : n = 7

Output : [0, 1, 1, 2, 3, 5, 8]

Algorithm :

- Initialize first two numbers a = 0, b = 1.
- Append 'a' to result list.
- Update a = b and b = a + b in each step.
- Repeat for n times and return result list.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def fibonacci(n):
    a, b = 0, 1
    result = []
    for _ in range(n):
        result.append(a)
        a, b = b, a + b
    return result

print(fibonacci(7))
# Output : [0, 1, 1, 2, 3, 5, 8]
```

Top 30 Python Coding Questions

★ Q7. Factorial (Recursive)

Input : n = 5

Output : 120

Algorithm :

- If n <= 1, return 1.
- Otherwise, return n * factorial(n - 1).
- Recursively reduce the problem.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
# Output : 120
```

★ Q8. Check if a Number is Prime

Input : n = 29

Output : True

Algorithm :

- If n < 2, return False.
- Check divisibility from 2 to $\sqrt{n}$.
- If any number divides n, return False.
- Otherwise, return True.

Time Complexity : $O(\sqrt{n})$

Space Complexity : O(1)

Code Solution :

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

print(is_prime(29))
# Output : True
```

★ Q9. Check if Two Strings are Anagrams

Input : s1 = "listen", s2 = "silent"

Output : True

Algorithm :

- Remove spaces and convert both strings to lowercase.
- Sort both strings.
- Compare sorted strings.
- If equal, return True, else False.

Time Complexity : O(n log n)

Space Complexity : O(n)

Code Solution :

```
def is_anagram(s1, s2):
    s1 = s1.replace(" ", "").lower()
    s2 = s2.replace(" ", "").lower()
    return sorted(s1) == sorted(s2)

print(is_anagram("listen", "silent"))
# Output : True
```

Top 30 Python Coding Questions

★ Q10. Remove Duplicates from a List (Preserve Order)

Input : [1, 3, 3, 2, 1, 5]

Output : [1, 3, 2, 5]

Algorithm :

- Use a set to track seen elements.
- Iterate through the list.
- If element not seen, add to result list and mark as seen.
- Return the result list.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def remove_duplicates(lst):
    seen = set()
    result = []
    for x in lst:
        if x not in seen:
            seen.add(x)
            result.append(x)
    return result

print(remove_duplicates([1,3,3,2,1,5]))
# Output : [1, 3, 2, 5]
```

★ Q11. Count Vowels in a String

Input : "Interview Preparation"

Output : 8

Algorithm :

- Convert string to lowercase.
- Check each character.
- If character is a vowel (aeiou), increment count.
- Return the count.

Time Complexity : O(n)

Space Complexity : O(1)

Code Solution :

```
def count_vowels(s):
    vowels = "aeiou"
    count = 0
    for ch in s.lower():
        if ch in vowels:
            count += 1
    return count

print(count_vowels("Interview Preparation"))
# Output : 8
```

★ Q12. Find Max/Min Without Built-in Functions

Input : [4, 9, 1, 7, 3]

Output : (9, 1)

Algorithm :

- Assume first element as max and min.
- Traverse the list from second element.
- Update max if current element is greater.
- Update min if current element is smaller.
- Return the max and min.

Time Complexity : O(n)

Space Complexity : O(1)

Code Solution :

```
def find_max_min(lst):
    mx = mn = lst[0]
    for x in lst[1:]:
        if x > mx:
            mx = x
        if x < mn:
            mn = x
    return mx, mn

print(find_max_min([4, 9, 1, 7, 3]))
# Output : (9, 1)
```

Top 30 Python Coding Questions

★ Q13. Merge Two Sorted Lists

Input : a = [1, 3, 5], b = [2, 4, 6]

Output : [1, 2, 3, 4, 5, 6]

Algorithm :

- Use two pointers i and j for both lists.
- Compare elements and add smaller element to result list.
- Move the pointer of the list from which element is taken.
- Add remaining elements of either list.
- Return the merged list.

Time Complexity : O(n + m)

Space Complexity : O(n + m)

Code Solution :

```
def merge_sorted(a, b):
    result, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1
    result.extend(a[i:])
    result.extend(b[j:])
    return result
print(merge_sorted([1, 3, 5], [2, 4, 6]))
# Output : [1, 2, 3, 4, 5, 6]
```

★ Q14. Flatten a Nested List

Input : [1, [2, 3, [4, 5]], 6]

Output : [1, 2, 3, 4, 5, 6]

Algorithm :

- Create an empty result list.
- Iterate through each item in the list.
- If item is a list, recursively flatten it and extend result.
- Else, append the item to result.
- Return the flattened list.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result
print(flatten([1, [2, 3, [4, 5]], 6]))
# Output : [1, 2, 3, 4, 5, 6]
```

★ Q15. List of Squares Using List Comprehension

Input : Range from 1 to 10

Output : [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

Algorithm :

- Use list comprehension.
- Iterate from 1 to 10.
- Square each number.
- Return the list of squares.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
squares = [x**2 for x in range(1, 11)]
print(squares)
# Output : [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Top 30 Python Coding Questions

★ Q16. Word Frequency Counter

Input : "the cat sat on the mat the cat ran"

Output : {"the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'ran': 1}

Algorithm :

- Convert text to lowercase and split into words.
- Use Counter from collections to count frequency.
- Return the Counter object.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
from collections import Counter

def word_frequency(text):
    return Counter(text.lower().split())

print(word_frequency(
    "the cat sat on the mat the cat ran"))

# Output : {"the': 3, 'cat': 2, 'sat': 1,
#           'on': 1, 'mat': 1, 'ran': 1}
```

★ Q17. Reverse Words in a Sentence

Input : "Python interview prep"

Output : "prep interview Python"

Algorithm :

- Split the sentence into words.
- Reverse the list of words.
- Join the words back with a space.
- Return the reversed sentence.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def reverse_words(sentence):
    return " ".join(sentence.split()[::-1])

print(reverse_words("Python interview prep"))

# Output : "prep interview Python"
```

★ Q18. Find Missing Number in an Array (1 to N)

Input : nums = [1, 2, 4, 5], n = 5

Output : 3

Algorithm :

- Calculate total sum of numbers from 1 to n using formula n*(n+1)//2.
- Calculate sum of elements in the list.
- Missing number = total sum - list sum.
- Return the missing number.

Time Complexity : O(n)

Space Complexity : O(1)

Code Solution :

```
def find_missing(nums, n):
    total = n * (n + 1) // 2
    return total - sum(nums)

print(find_missing([1, 2, 4, 5], 5))
# Output : 3
```

Top 30 Python Coding Questions

★ Q19. Binary Search

Input : arr = [1, 3, 5, 7, 9, 11], target = 7

Output : 3

Algorithm :

- Initialize low and high pointers.
- Find middle element.
- Compare middle element with target.
- If equal, return index.
- If target is smaller, search in left half.
- If target is larger, search in right half.
- Repeat until found or range is invalid.

Time Complexity : O(log n)

Space Complexity : O(1)

Code Solution :

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

print(binary_search([1, 3, 5, 7, 9, 11], 7))
# Output : 3
```

★ Q20. Bubble Sort

Input : arr = [5, 2, 9, 1, 5, 6]

Output : [1, 2, 5, 5, 6, 9]

Algorithm :

- Compare adjacent elements.
- If first is greater, swap them.
- After each pass, largest element moves to the end.
- Repeat for remaining unsorted part.
- Return the sorted list.

Time Complexity : $O(n^2)$

Space Complexity : O(1)

Code Solution :

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

print(bubble_sort([5, 2, 9, 1, 5, 6]))
# Output : [1, 2, 5, 5, 6, 9]
```

★ Q21. Quick Sort

Input : arr = [9, 3, 7, 1, 8, 2]

Output : [1, 2, 3, 7, 8, 9]

Algorithm :

- Choose a pivot element.
- Partition list into three parts: less than, equal to, and greater than pivot.
- Recursively apply quick sort on left and right parts.
- Combine sorted parts and return.

Time Complexity : O(n log n) (Average)

Space Complexity : O(log n) (Recursion Stack)

Code Solution :

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)

print(quick_sort([9, 3, 7, 1, 8, 2]))
# Output : [1, 2, 3, 7, 8, 9]
```

Top 30 Python Coding Questions

★ Q22. Find the Second Largest Number in a List

Input : [4, 9, 1, 9, 7]

Output : 7

Algorithm :

- Remove duplicates using a set.
- Convert set back to list and sort.
- If list has less than 2 elements, return None.
- Return second last element.

Time Complexity : O(n log n)

Space Complexity : O(n)

Code Solution :

```
def second_largest(lst):
    unique = list(set(lst))
    if len(unique) < 2:
        return None
    unique.sort()
    return unique[-2]

print(second_largest([4, 9, 1, 9, 7]))
# Output : 7
```

★ Q23. Check Balanced Parentheses

Input : "{[()]}"

Output : True

Algorithm :

- Use a stack to store opening brackets.
- If closing bracket is found, pop from stack and check if it matches.
- If mismatch or stack is empty, return False.
- After traversal, if stack is empty, return True.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def is_balanced(s):
    stack = []
    pairs = {')':'(', ']':'[', '}':'{'}
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

print(is_balanced("{[()]}"))   # Output : True
print(is_balanced("{[()]}"))   # Output : False
```

★ Q24. Implement atoi (String to Integer)

Input : " -42"

Output : -42

Algorithm :

- Remove leading/trailing spaces.
- Check for sign (+/-).
- Iterate and build number from digits.
- Stop at first non-digit character.
- Return number with sign.

Time Complexity : O(n)

Space Complexity : O(1)

Code Solution :

```
def my_atoi(s):
    s = s.strip()
    if not s:
        return 0
    sign, i = 1, 0
    if s[0] in "+-":
        sign = -1 if s[0] == '-' else 1
        i = 1
    num = 0
    while i < len(s) and s[i].isdigit():
        num = num * 10 + int(s[i])
        i += 1
    return sign * num

print(my_atoi("  -42"))   # Output : -42
```

Top 30 Python Coding Questions

★ Q25. Longest Common Prefix

Input : ["flower", "flow", "flight"]
Output : "fl"

Algorithm :

- Take first string as prefix.
- Compare with remaining strings.
- Reduce prefix until it matches all strings.
- Return the longest common prefix.

Time Complexity : O(S)
Space Complexity : O(1)

Code Solution :

```
def longest_common_prefix(strs):
    if not strs:
        return ""
    prefix = strs[0]
    for s in strs[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
            if not prefix:
                return ""
    return prefix

print(longest_common_prefix(["flower", "flow", "flight"]))
# Output : "fl"
```

★ Q26. Rotate an Array by K Positions

Input : [1, 2, 3, 4, 5], k = 2
Output : [4, 5, 1, 2, 3]

Algorithm :

- Find effective rotations: k % n.
- Take last k elements and put at front.
- Append remaining elements after that.
- Return the rotated array.

Time Complexity : O(n)
Space Complexity : O(n)

Code Solution :

```
def rotate_array(arr, k):
    n = len(arr)
    k = k % n
    return arr[-k:] + arr[:-k]

print(rotate_array([1, 2, 3, 4, 5], 2))
# Output : [4, 5, 1, 2, 3]
```

★ Q27. Find Intersection of Two Lists

Input : [1, 2, 3, 4], [3, 4, 5, 6]
Output : [3, 4] (order may vary)

Algorithm :

- Convert both lists to sets.
- Find intersection using & operator.
- Convert result back to list.
- Return the intersection list.

Time Complexity : O(n + m)
Space Complexity : O(n + m)

Code Solution :

```
def intersection(a, b):
    return list(set(a) & set(b))

print(intersection([1, 2, 3, 4], [3, 4, 5, 6]))
# Output : [3, 4]
```

Top 30 Python Coding Questions

★ Q28. Remove Duplicates from Sorted Array

Input : nums = [1, 1, 2, 2, 3, 3, 4]

Output : 4, nums = [1, 2, 3, 4, _, _, _]

Algorithm :

- Use two pointers i and j.
- i tracks the position of unique element.
- j iterates through the array.
- If nums[j] is different from nums[i], move i forward and update nums[i].
- Return (i + 1) as count of unique elements.

Time Complexity : O(n)

Space Complexity : O(1)

Code Solution :

```
def remove_duplicates(nums):
    if not nums:
        return 0
    i = 0
    for j in range(1, len(nums)):
        if nums[j] != nums[i]:
            i += 1
            nums[i] = nums[j]
    return i + 1

nums = [1, 1, 2, 2, 3, 3, 4]
k = remove_duplicates(nums)
print(k, nums[:k] + ['_']*(len(nums)-k))
# Output : 4  [1, 2, 3, 4, '_', '_', '_']
```

★ Q29. Two Sum

Input : nums = [2, 7, 11, 15], target = 9

Output : [0, 1]

Algorithm :

- Use a hash map to store number and its index.
- For each number, check if (target - number) exists in the map.
- If found, return both indices.
- Otherwise, store the number and its index.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
nums = [2, 7, 11, 15], target = 9
print(two_sum(nums, target))
# Output : [0, 1]
```

★ Q30. Subarray Sum Equals K

Input : nums = [1, 1, 1], k = 2

Output : 2

Algorithm :

- Use prefix sum and a hash map.
- Store frequency of each prefix sum.
- For each prefix sum, check if (prefix - k) exists in the map.
- Add the frequency to result.
- Update the map with current prefix sum.

Time Complexity : O(n)

Space Complexity : O(n)

Code Solution :

```
def subarray_sum(nums, k):
    prefix = 0
    count = {0: 1}
    res = 0
    for num in nums:
        prefix += num
        if prefix - k in count:
            res += count[prefix - k]
        count[prefix] = count.get(prefix, 0) + 1
    return res
nums = [1, 1, 1], k = 2
print(subarray_sum(nums, k))
# Output : 2
```